Listing 11.10 Using attributes to describe your API

```
using System.Collections.Concurrent;

WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

WebApplication app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

var _fruit = new ConcurrentDictionary<string, Fruit>();

app.MapGet("/fruit/{id}",
    [EndpointName("GetFruit")]                                          ❶
    [EndpointSummary("Fetches a fruit")]                                ❶
    [EndpointDescription("Fetches a fruit by id, or returns 404" +      ❶
        " if no fruit with the ID exists")]                             ❶
    [ProducesResponseType(typeof(Fruit), 200)]                          ❶
    [ProducesResponseType(typeof(HttpValidationProblemDetails), 404,    ❶
        "application/problem+json")]                                    ❶
    [Tags("fruit")]                                                     ❶
    (string id) =>
        _fruit.TryGetValue(id, out var fruit)
            ? TypedResults.Ok(fruit)
            : Results.Problem(statusCode: 404))
    .WithOpenApi(o =>
    {
        o.Parameters[0].Description = "The id of the fruit to fetch";
        return o;
    });

app.Run();
record Fruit(string Name, int Stock);
```

❶ You can use attributes instead of fluent method calls.

Whether you think listing 11.10 is better than listing 11.9 is largely a matter of taste, but the reality is that neither is particularly elegant. In both cases the metadata significantly obscures the intent of the API, so it’s important to consider which metadata is worth adding and which is unnecessary noise. That balance may shift depending on who your audience is (internal or external customers), how mature your API is, and how much you can extract to helper functions.